Drew University - CSCI 260 - Fall 2024
LC-3 Assembly Language Project

Project Info.: Due on 11/11/2024 (M) by 11:59pm, 15% of the total grade
Submission: On Moodle by everyone; see below.

Background: The *Caesar Cipher*, said to be invented by Julius Caesar and was used by the ancient Roman Legion, is the earliest known method of "encrypting" a message (the "plaintext") to something that is unintelligible (the "cipertext") by using a secret number (the "key") which is only known to the sender (Alice) and the expected receiver (Bob) of that message, but no one else.

The cipher works as such: first of all, Alice performs an "encryption" using the shared key. And after the encrypted message is delivered to Bob. Now, when Bob receives the message, he will apply the key to the cipher to perform a "decryption" and recover the original message fairly quickly and easily. However, if the message somehow was obtained by an adversary (Cate) in the middle, since Cate does not have the key, it will take her quite some effort to find out the original message. As well as that, if Cate altered the ciphertext, Bob will for sure know that, as the decrypted text from that altered one will most likely have no meanings.

In this project, you are asked to implement such a cipher with the said encryption and decryption functionalities on text and key being typed in from the keyboard.

Description: The flow of the program looks like this:

1. At the beginning of the program, you will be asked to type in either "E"/"D" for calling the corresponding encryption/decryption protocol via the prompt "Please select mode [E/D]: ".
2. Then, in either case, you will be asked to input a string via the prompt "Please enter the plain/ciphertext ", where the string is terminated by an enter, or x0A in LC-3.
3. Then, similarly, you will be asked to type in the key, a number ranging from 0-9, via the prompt "Please enter the key (0-9): ".
4. Lastly, depending on the initial selection, the system will print to the console either the encrypted/decrypted message with promopt "The encrypted/decrpted message is: ... ...," where the dots represent the ciphertext/plaintext.

To give a specific use case scenario: We choose encryption with plaintext "HelloWorld" (let's focus on letters only, upper and lower cases) with key being 5. After the program execution, the ciphertext with prompt "The encrypted message is: MjqqtBtwqi" should be printed to the console.

Andy Siyuan Feng, Drew University, Fall 2024

Algorithms: Notice how in the example above, each letter in the plaintext is *shifted forward* by 5 alphabetical places? Turns out Caesar Cipher is a type of "Shift Cipher" where the shifting amount is the key. This is the sort of behavior we want to achieve for the encryption function. One more thing, notice how "W" is shifted 3 places to "Z", and wrapped around, and is shifted 2 more places to "B". This wrapping behavior is also desired. (Think about how can we achieve this when using ASCII representation.)

Conversely, if we want to decrypt the ciphertext "MjqqtBtwqi", or *shift backward*, it is most likely, only when the key is 5, the one used for encryption, the plaintext we recovered will make any sense. This is the behavior for the decryption function we want to achieve. Note that, for decryption, we also have the wrapping behaviour.

During the mode selection, if the user typed something other than "E"/"D", then the program will display an error message "Mode Selection Error: please select mode [E/D]: " and wait for the user to enter again.

We can assume that the text input are from the correct domain.

To simplify the program a little, during the key input, assume the user will type a key value that is between 0 and 9.

How to proceed: Work in group of 3-4 people and divide the project into 3-4 modules, input/output handling and function calls, the encryption function, and the decryption function, so each person can be in charge of completing one part before they are integrated into a whole program.

Start the program at x3000 location.

Store the mode at x4000 location. Store the key at x4001 location.

Assume the input text is at most 20 characters long, so you can reserve 20+1 (with a null terminator) locations from x4002 to x4016.

Similarly, reserve 20+1 (with a null terminator) locations from x4017 to x402B for the output.

You should provide enough comments to indicate what each line/section of code is trying to achieve. A well-commented submission will received 2% extra credits.

Submission: You must submit a .txt file that contains an executable, well-commented Assembly code for the program. Put group members names in the comment on the very top of that file. Optionally, you can submit a .pdf file containing any development documentations you want to share. By showing good effort via that document will earn you up to 2% extra credits.